ChatGPT

Project Overview

We will build an audio-processing application that takes sound input (e.g. a user recording) and uses **deep learning** to analyze or classify it. In practice this means converting raw audio waves into numerical features (like spectrograms) and training a neural network on them. Modern audio AI systems (speech recognizers, sound classifiers, etc.) typically require careful data preprocessing, augmentation and feature extraction before feeding data into a deep CNN/RNN model [1] [2]. For example, one guide explains that the goal is to "convert a raw waveform … into a spectrogram, which is a visual image" and then use a convolutional network on that image [2]. In our project, the **end goal** might be to classify incoming sounds (e.g. recognize an animal call or a keyword) using a deep model trained on example recordings. Because mentors insist on a deep learning solution, we will emphasize neural nets (CNNs/RNNs) rather than only classical ML.

Prerequisites

Before coding, ensure the team has the following background and tools:

- **Programming & Libraries:** Familiarity with Python is essential. We'll use ML/DL frameworks like **TensorFlow/Keras or PyTorch** for modeling [3] [4]. Required Python libraries include numerical and data tools (e.g. NumPy, pandas, Matplotlib) and audio libraries like **Librosa** or **TorchAudio** for loading and processing sound [5]. For example, one tutorial uses PyTorch and even lists installing `pandas` and `torchsummary` via `pip` [3].
- **Machine Learning & Deep Learning:** Basic understanding of neural networks (CNNs, RNNs), training procedures, loss functions, and overfitting. You should be comfortable splitting data, monitoring training/validation accuracy, and tuning hyperparameters. One digital-ocean guide explicitly recommends familiarity with *"deep learning frameworks" and other basic concepts* before starting [4].
- **Audio/Signal Knowledge:** Basic signal processing ideas help (sampling rate, time vs frequency domain, noise). For example, know that audio is often stored as PCM (Pulse Code Modulation) in files. A WAV file is an uncompressed PCM format, whereas MP3 is a compressed lossy format [6]. We will usually use WAV (16-bit, 16kHz or 44.1kHz) for consistency [6]. We should also understand concepts like Fourier transforms and mel scales at a high level for feature extraction.
- **Hardware/Environment:** It's ideal to have access to a GPU or a cloud service (e.g. Colab). Ensure you have a development environment set up (Jupyter/VS Code) and version control. Install any required libraries (e.g. `pip install librosa`, `pip install torch`, etc.).

Project Modules and Team Roles

We'll split the project into **4 modules**, one per team member, each focusing on a key stage of the pipeline:

1. **Data Collection & Cleaning:**
2. **Tasks:** Gather and organize audio samples. This could mean downloading existing datasets (e.g. UrbanSound8K, ESC-50, Kaggle challenges) or recording your own data. For instance, the

UrbanSound8K dataset has 8,732 labeled urban clips across 10 classes [7]. Store the audio files (use WAV format if possible) and create labels.

3. **Preprocessing:** Perform initial cleanup. Remove silent or corrupted clips (e.g. those with near-zero amplitude) [8]. Normalize volume levels so all clips have similar loudness (one method is to scale the peak amplitude to 0.99 to avoid clipping) [8]. Explore basic dataset statistics (clip lengths, sample rates) to catch anomalies. Output a clean directory structure (e.g. folders per class with uniformly formatted WAV files).

4. **Feature Engineering & Normalization:**

5. **Tasks:** Convert raw waveforms into numeric feature vectors usable by a model. First, load each WAV file (e.g. with Librosa or torchaudio [5]) to get a 1D array of samples and a sample rate. Resample all clips to a common rate (e.g. 16 kHz) and mix stereo to mono if needed. Trim or pad each clip to a fixed duration (for example, cut longer clips and pad shorter ones to 5 seconds) [9] so that inputs have uniform length.
6. **Feature Extraction:** Compute time-frequency features. The most common approach is to calculate a **spectrogram** or **mel-spectrogram** from the audio. A spectrogram is a visual representation of frequency content over time, obtained by applying short-time Fourier transforms to sliding windows of the waveform [10]. A mel-spectrogram is a spectrogram with its frequency axis warped to the Mel scale (more aligned with human hearing) [10]. For example, one guide outlines extracting mel-spectrograms and converting amplitudes to dB [11]. The result is a 2D image-like array which we can treat as input to a CNN. Other possible features include **MFCCs** (Mel-Frequency Cepstral Coefficients), which are a compressed representation of the spectrum [10].

7. **Normalization:** Scale or normalize the features. For spectrograms or MFCCs, we often apply log scaling and standardization so that each feature has zero mean/unit variance. We already normalized raw audio amplitude in step 1; now ensure each feature map is on a consistent scale. Data augmentation can also be done here: for example, randomly shift the audio in time, pitch-shift, add background noise or stretch time to increase diversity [12]. This helps the model generalize.

8. **Model Design & Training:**

9. **Tasks:** Define and train the deep learning model. The typical architecture for audio classification is a Convolutional Neural Network that takes the spectrogram (treated as an image) as input [2]. You might start with a simple CNN (a few convolutional + pooling layers, followed by dense layers). For improved performance, consider more advanced architectures: for instance, combining a CNN with a recurrent layer (like an LSTM or GRU) to capture temporal sequences [13]. In one study of bird-call classification, a CNN was used to process spectrogram slices and an RNN was used to integrate the temporal dimension, yielding the best accuracy [13].

10. **Training:** Split data into training/validation sets. Use a suitable loss (e.g. cross-entropy for classification) and optimizer (Adam is common). Monitor performance on a hold-out validation set. If classes are unbalanced, consider weighted loss or further data balancing. Use regularization (dropout, early stopping) to avoid overfitting. Make sure the model is *deep* enough – simple shallow models tend to underfit; mentors expect a proper neural net, not just logistic regression or decision trees. You can experiment with depth, number of filters, or even pretrained models if applicable. Track metrics (accuracy, F1-score) as you train.

11. **Evaluation, Pipeline & Deployment:**

12. **Tasks:** Assemble the end-to-end pipeline and evaluate thoroughly. Write code that ties the data loader, feature extractor, and model trainer into one workflow. For example, use framework data pipelines (like `tf.data` or `torch.utils.data.Dataset`) to automate feeding audio → features → model. After training, evaluate on a separate test set to compute final accuracy. Generate reports or visualizations (confusion matrix, training curves). Ensure all code is documented and reproducible.
13. **Deliverables:** Prepare final deliverables: trained model files, inference scripts, and a report or presentation. Optionally, create a simple demo where a user can input a sound and the system outputs a prediction. The **quality** of the model should include robustness (works on noisy inputs), reasonable inference speed, and high accuracy.

Working with Audio Data

When coding, remember: **audio is just data** until processed. We store recordings in files (typically WAV) and load them into Python. For example, a WAV file contains PCM samples at a certain *sample rate* [6]. We use an audio library like Librosa (`librosa.load('file.wav')`) which returns a NumPy array of samples plus the sampling rate [5]. It's common to then convert stereo to mono (averaging channels) and resample (e.g. to 16,000 Hz) so all clips share the same rate.

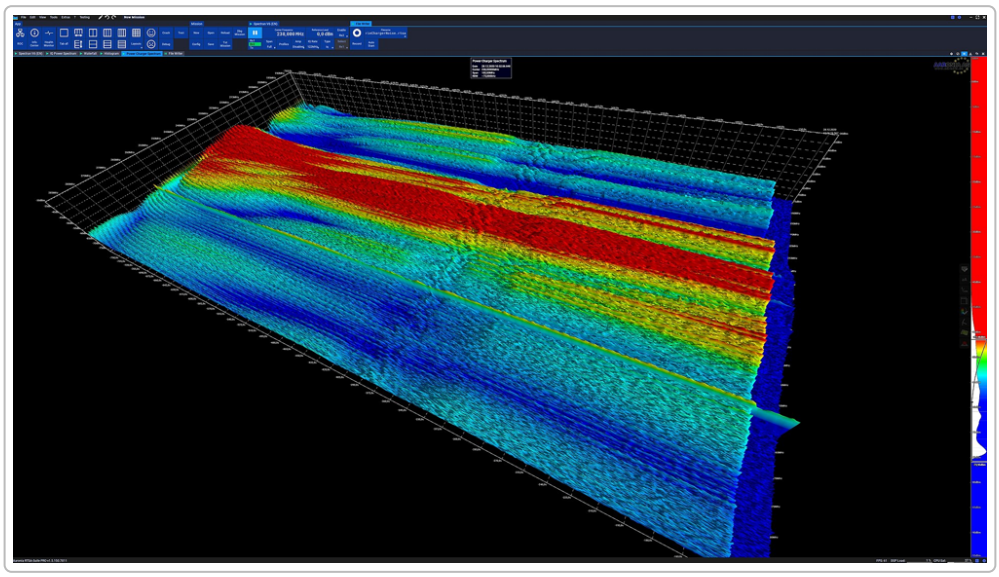

Image: A mel-spectrogram (time-frequency heatmap) extracted from an audio clip. Once we have a clean waveform array (float samples), we extract features. A mel-spectrogram (shown above) is obtained by slicing the audio into short windows, computing a Fourier transform on each window, and mapping frequencies to the Mel scale [10]. Deep-learning pipelines often feed these spectrograms directly into CNNs [2]. In other words, the raw audio (time domain) is converted to an "image" where one axis is time and the other is frequency [10]. Alternatively, MFCC vectors could be used. The key is: *"Deep Learning approach considers unstructured audio representations such as the spectrogram or MFCCs. It extracts the patterns on its own"* [10].

Before feeding features to the model, normalize them. We already normalized volume; now we typically take log-magnitude of the spectrogram and scale it. Also ensure every example has the same shape (if clips

vary in length, trim or pad them). For example, one guide trims silence and fixes length to 5 seconds [9]. In practice, that means cutting or zero-padding so each feature array is e.g. `[1 × 128 × 156]` (1 channel, 128 mel bins, 156 time frames). This consistent sizing is crucial for batch training.

Finally, use appropriate **libraries** and tools: as noted, Librosa and PyTorch's TorchAudio are designed for audio preprocessing [5]. They can compute spectrograms and augment audio easily. Combine them with deep learning libraries (TensorFlow/Keras or PyTorch) to build your model architecture.

Throughout, **document your process**: note which datasets you use (cite sources or links), and keep your code modular (one module reads data, one extracts features, one trains model, etc.). Use version control and comment your code. By following this plan – clear modules, a thorough pipeline from raw audio to deep model, and sound coding practices – your team will produce a polished, high-scoring project that goes well beyond a simple ML approach [1] [2].

---

[1] [8] [9] [12] Building a Complete Audio Classification Pipeline Using Deep Learning: From Raw Audio to Mel Spectrogram CNNs | by Omar Rehan | Medium
https://medium.com/@ai.omar.rehan/building-a-complete-audio-classification-pipeline-using-deep-learning-from-raw-audio-to-mel-9894bd438d85

[2] [4] Audio Classification with Deep Learning | DigitalOcean
https://www.digitalocean.com/community/tutorials/audio-classification-with-deep-learning

[3] [7] [11] Guide to Audio Classification Using Deep Learning
https://www.analyticsvidhya.com/blog/2022/04/guide-to-audio-classification-using-deep-learning/

[5] [10] Audio Feature Extraction
https://devopedia.org/audio-feature-extraction

[6] Learning roadmap for the audio signal processing beginner - Signal Processing Stack Exchange
https://dsp.stackexchange.com/questions/9194/learning-roadmap-for-the-audio-signal-processing-beginner

[13] Comparing recurrent convolutional neural networks for large scale bird species classification - PMC
https://pmc.ncbi.nlm.nih.gov/articles/PMC8385065/